

## Problema 1 – VAIUNTO: Sistema de Caronas Compartilhadas

Calendário

| Semana | Data  | Grupo Tutorial                     |
|--------|-------|------------------------------------|
| 1      | 18/08 | Sessão 1: Apresentação do Problema |
| 2      | 20/08 | Sessão 2: Desenvolvimento          |
| 3      | 25/08 | Sessão 3: Tutorial                 |
| 4      | 27/08 | Sessão 4: Desenvolvimento          |
| 5      | 01/09 | Sessão 5: Tutorial                 |
| 6      | 03/09 | Sessão 6: Desenvolvimento          |
| 7      | 08/09 | Sessão 7: Tutorial                 |
| 8      | 10/09 | Sessão 8: Desenvolvimento          |
| 9      | 15/09 | Sessão 9: Desenvolvimento          |
| 10     | 17/09 | Entrega do produto e apresentações |
| 11     | 22/09 | Final das apresentações            |

### Contexto

O transporte rodoviário intermunicipal e interestadual no Brasil convive com uma contradição conhecida: enquanto muitos viajantes enfrentam custo elevado e oferta limitada de horários, milhares de automóveis percorrem diariamente as mesmas rodovias com a maior parte de seus assentos vazios. Cada assento ocioso representa combustível, pedágio e desgaste do veículo pagos integralmente por uma única pessoa, além de um carro a mais em circulação para transportar o mesmo número de passageiros.

Foi a partir dessa ociosidade que surgiram as plataformas de carona compartilhada de média e longa distância, das quais o BlaBlaCar é o exemplo mais conhecido internacionalmente. O princípio é simples: um motorista que já faria determinado percurso anuncia os assentos livres de seu veículo, e passageiros que precisam se deslocar entre as mesmas cidades reservam esses assentos, dividindo os custos da viagem. Diferentemente dos aplicativos de transporte urbano, a rota não é definida pelo passageiro, mas pelo motorista: a plataforma não cria a viagem, apenas descobre e coordena os encontros possíveis entre uma oferta que já existe e uma demanda dispersa. Toda a dificuldade técnica está justamente nessa coordenação, feita de forma automatizada, remota e simultânea entre milhares de usuários que nunca se encontraram.

### Problema

Uma startup de mobilidade deseja lançar um serviço de caronas compartilhadas para viagens de média e longa distância. Atualmente, a empresa mantém um servidor central, onde os motoristas publicam as caronas que pretendem realizar e os passageiros consultam e reservam assentos pela Internet, sem qualquer intermediação humana.

Ao publicar uma carona, o motorista informa a sua rota — uma sequência ordenada de cidades pelas quais ele passará —, a data e o horário de partida, a quantidade de assentos livres em seu veículo e o valor cobrado por trecho. Como o veículo passa por cidades intermediárias, um passageiro pode embarcar e desembarcar em qualquer par de cidades da rota, o que significa que a disponibilidade de assentos deve ser controlada por trecho, e não pela carona como um todo: um assento ocupado entre a primeira e a segunda cidade da rota permanece livre para os trechos seguintes.

Do outro lado, o passageiro informa a cidade de origem, a cidade de destino e a data desejada, e o servidor deve responder com os itinerários possíveis. Um itinerário pode ser formado pelos trechos de uma única carona ou pela combinação de trechos de caronas de motoristas diferentes. Por exemplo, um passageiro que deseja ir de Salvador a Vitória da Conquista pode não encontrar nenhum motorista fazendo o percurso completo, mas pode montar a viagem reservando o trecho de Salvador a Feira de Santana na carona do motorista A e, em seguida, o trecho de Feira de Santana a Vitória da Conquista na carona do motorista B.

A confirmação de um itinerário deve ser atômica: ou todos os trechos escolhidos são reservados para o passageiro, ou nenhum deles é, pois de nada adianta garantir a primeira metade de uma viagem e ficar sem a segunda. Como diversos passageiros consultam e reservam ao mesmo tempo, um assento anunciado como disponível pode deixar de existir entre a consulta e a confirmação. O passageiro que confirmar primeiro deve manter a preferência sobre os assentos, cabendo aos demais desistir ou escolher outro itinerário. Sob nenhuma hipótese o sistema pode confirmar o mesmo assento de um mesmo trecho para dois passageiros distintos, nem deixar assentos permanentemente bloqueados por uma reserva que nunca foi concluída.

Ficou decidido que o sistema deve ser implementado sobre o subsistema de rede TCP/IP, onde o protocolo de comunicação deve ser definido de acordo com uma API especificada pela sua equipe de desenvolvimento. Esta API deverá ser programada em qualquer linguagem com suporte à API Socket básica do TCP/IP, visando oferecer as funcionalidades necessárias para a comunicação entre os clientes e o servidor.

Tão importante quanto o programa a ser desenvolvido é o poder de sua argumentação em demonstrar a solução. Por isso, deve-se preparar um relatório, formato SBC, no máximo, 8 páginas, demonstrando o embasamento teórico utilizado para a solução do problema e as escolhas técnicas necessários para o desenvolvimento do software. Além desta descrição será fundamental para a aceitação do seu produto.

#### **Produto**

Além do servidor central que mantém o estado das caronas e das reservas, os seguintes componentes do produto devem ser implementados e apresentados pelo aluno:

1. Um cliente motorista, com interface (gráfica ou de terminal), que permita autenticar-se, publicar uma carona informando rota, data, assentos e preço por trecho, consultar as caronas já publicadas e os passageiros confirmados em cada trecho, e cancelar uma carona;
2. Um cliente passageiro, com interface (gráfica ou de terminal), que permita autenticar-se, buscar itinerários entre uma origem e um destino em uma data, confirmar a reserva de um itinerário composto por um ou mais trechos, e consultar ou cancelar suas reservas;
3. A especificação completa do protocolo de aplicação definido pela equipe, documentada no repositório, contendo o formato das mensagens, os campos de cada operação, o fluxo de conexão e desconexão e exemplos concretos de mensagens trocadas;
4. Um teste de software automatizado que submeta o servidor a múltiplos clientes simultâneos disputando os mesmos trechos, verificando que nenhum assento é vendido duas vezes, que nenhum itinerário é confirmado pela metade e medindo o tempo de resposta do sistema sob carga.

#### **Restrições**

- Os componentes backend do sistema devem ser desenvolvidos e testados por meio de contêineres Docker, para facilitar a execução de mais de uma instância;
- A comunicação entre os clientes e o servidor deve ser implementada usando a interface de socket nativa do TCP/IP;

- Por questões comerciais, nenhum framework de troca de mensagens ou de chamada remota de procedimento deve ser usado para implementar a comunicação entre os clientes e o servidor;
- Neste primeiro protótipo, a empresa manterá um único servidor central, responsável por todo o estado do sistema. Nenhuma réplica ou servidor secundário deve ser empregado;
- O controle de concorrência sobre os assentos é responsabilidade da solução desenvolvida, não sendo permitido delegá-lo a um sistema gerenciador de banco de dados ou a qualquer serviço externo de coordenação;
- O servidor deve atender vários clientes simultaneamente e permanecer disponível durante toda a apresentação, onde a queda ou o encerramento abrupto de um cliente não pode interromper o serviço nem corromper o estado das reservas;
- Os dados trocados devem ser codificados em uma representação intermediária bem-definida, de modo que clientes e servidor escritos em linguagens diferentes possam interoperar, cabendo ao receptor validar e descartar mensagens malformadas;
- Para uma emulação realista do cenário proposto, os contêineres do servidor e dos clientes devem ser executados em computadores distintos no laboratório.

#### Nossas Regras

- Os alunos devem implementar o problema individualmente.
- O prazo final de entrega do trabalho e apresentação será dia 17/09/2026.
- O código fonte deve ser entregue devidamente comentado (relatório) por meio da plataforma GitHub.
- Os alunos devem elaborar um relatório no formato padrão SBC devidamente referenciado.
- Será estabelecida uma agenda, onde o aluno terá 20 minutos para executar e realizar a sua apresentação. Durante a apresentação, o tutor realizará arguições ao aluno.
- O sistema deve ser desenvolvido, testado e apresentado no Laboratório de Redes e Sistemas Distribuídos (LARSID) ou LADICA.

#### Observações

- Trabalhos entregues fora do prazo serão penalizados com 20% do valor da nota + 5% por dia de atraso, dentro da mesma semana da entrega final.
- Trabalhos copiados da INTERNET ou de qualquer outra fonte e trabalhos idênticos terão nota ZERO.
- As informações para solução do problema podem ser ALTERADAS no decorrer das sessões.

#### Avaliação

A nota final será a composição de 03 notas:

1. Desempenho individual (30%).
2. Relatório formato SBC (20%).
3. Produto no GitHub (com README) - 50%.